

Fabric-as-Code

Deploy Microsoft Fabric end-to-end — from nothing to a running platform

Azure CLI · Bicep · Fabric REST APIs

The problem: two control planes

- Microsoft Fabric has two separate control planes — the portal covers neither repeatably:
 - Azure — the Fabric capacity (Microsoft.Fabric/capacities), the billable compute
 - Fabric — workspaces and items: Lakehouse, Warehouse, Notebook, Pipeline...
 - Data — objects inside an item, e.g. stored procedures in a Warehouse
- ARM/portal cannot create workspaces or items — only the Fabric REST API can.

What this repo does

- One config file drives a fully working environment:
 - Provision a resource group + Fabric capacity (Bicep)
 - Create a Fabric workspace (REST)
 - Assign the workspace to the capacity (REST)
 - Deploy items — Lakehouse, Warehouse, Notebook, Data Pipeline (REST)
 - Deploy stored procedures into the Warehouse (T-SQL)
 - Optional: connect the workspace to Git
- Every step is idempotent and parameter-driven — re-runnable and safe.

Repeatable by design

- All tenant / subscription / secret values live in a git-ignored .env
- Duplicate .env per environment: .env.dev, .env.customerA, ...
- Same scripts, different config → identical environment in any tenant / RG
- Service-principal auth for CI/CD and multi-tenant automation
- PowerShell and bash versions of every script
- Public repo: contains no secrets

Fabric — workspace contents

Every item created via the Fabric REST API.

 **Fabric-as-Code Demo** 

Workspace provisioned end-to-end by the fabric-as-code repo.

+ New item

 New folder

 Import ▾

 Migrate

	Name	Status	Type	Task	Owner	Refresh
	lh_demo		Lakehouse	—	MOD Administrator	—
	lh_demo		SQL analytics endpoint	—	MOD Administrator	—
	nb_demo_load		Notebook	—	MOD Administrator	—
	pl_demo_ingest		Pipeline	—	MOD Administrator	—
	wh_demo		Warehouse	—	MOD Administrator	—

Fabric — Lakehouse with data

orders_bronze Delta table, written by the notebook run (5 rows).

lh_demo

Search

Home

Materialized lake views

Get data

New semantic model

Add to data agent

Manage OneLake security

Update all variables

For optimal refresh to work efficiently, enable the delta CDF property on both source tables and materialized lake views used as source.

A SQL analytics endpoint for SQL querying was created with this item.

Explorer

Add lakehouses

lh_demo

Tables

orders_bronze

Files

orders_bronze

Table view

	12L order_id	12L customer_id	12L order_date	12 amount	ABC status
1	1	101	2026-01-05	120.5	shipped
2	2	102	2026-01-06	75	shipped
3	3	101	2026-01-07	240	pending
4	4	103	2026-01-08	18.99	cancelled
5	5	104	2026-01-09	999.95	shipped

Analyze your data

Explore your lakehouse data in a notebook, SQL analytics endpoint, or eventhouse endpoint.

Analyze data with

Fabric — Notebook

nb_demo_load, bound to the Lakehouse, writes the demo dataset with PySpark.

The screenshot displays the Microsoft Fabric Notebook interface. At the top, the notebook name 'nb_demo_load' is shown in a dropdown menu. Below this is a search bar and a navigation bar with tabs for 'Home', 'Edit', 'AI tools', 'Run', and 'View'. A secondary bar contains icons for file operations, a 'Run all' button, a 'Connect' button, and a dropdown for 'PySpark (Python)'. The 'Environment' section shows 'Workspace default'. To the right are icons for 'Data Wrangler', a share icon, a refresh icon, and 'Copilot'. On the left, an 'Explorer' pane shows 'Data items', 'Resources', and 'Connections' tabs, with a search bar and a '+ Add data items' button. Below this, a tree view shows 'OneLake'. The main area features a warning message: 'Other people in your organization may have access to notebooks and Spark job definitions in this workspace. Carefully review this item before running it.' The notebook title 'Demo load notebook' is prominently displayed, followed by its description: 'Provisioned by fabric-as-code. Writes a small demo dataset into the attached Lakehouse as a Delta table.' Two code blocks are visible. The first block contains PySpark code to create a DataFrame from a list of rows. The second block contains code to write the DataFrame to a Delta table and print a confirmation message.

nb_demo_load ▾

Search

Home Edit AI tools Run View

📁 📄 ⚙️ ▶ Run all ▾ 🔗 Connect ▾ 📄 PySpark (Python) ▾ Environment Workspace default ▾ 🛠️ 🔄 🔄 🔄 Copilot

Explorer <<

📁 Data items 📄 Resources 📁 Connections

+ Add data items 🔍

▾ OneLake

ⓘ Other people in your organization may have access to notebooks and Spark job definitions in this workspace. Carefully review this item before running it.

Demo load notebook

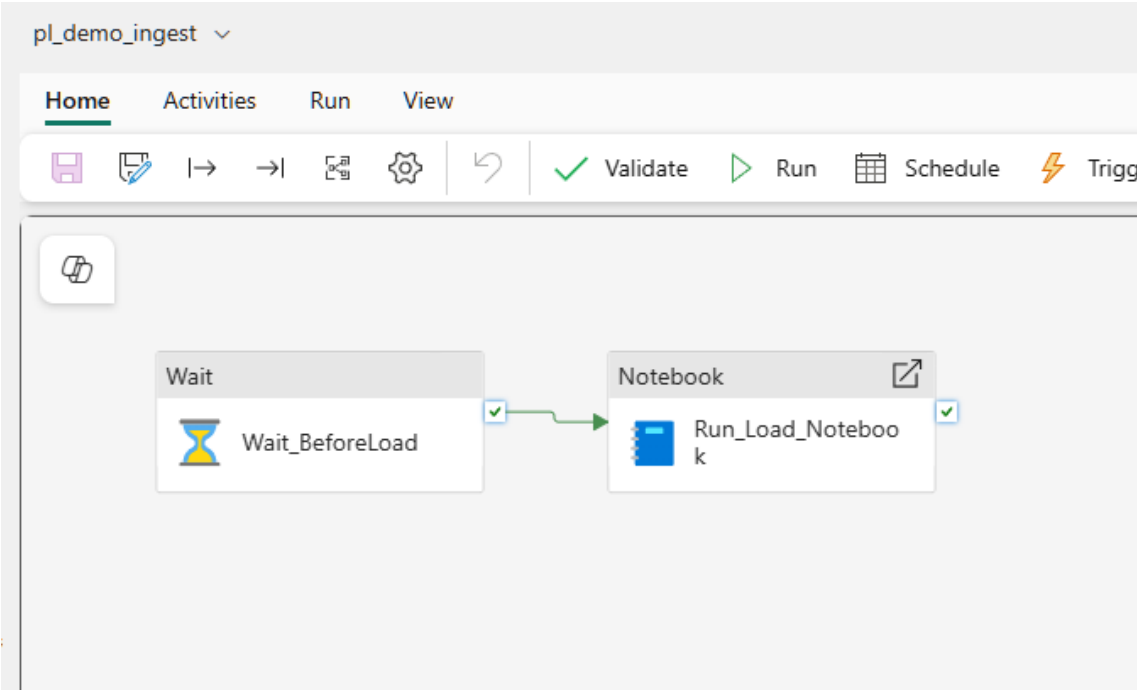
Provisioned by **fabric-as-code**. Writes a small demo dataset into the attached Lakehouse as a Delta table.

```
1 from pyspark.sql import Row
2 from datetime import date
3
4 rows = [
5     Row(order_id=1, customer_id=101, order_date=date(2026, 1, 5), amount=120.50, status='shipped'),
6     Row(order_id=2, customer_id=102, order_date=date(2026, 1, 6), amount=75.00, status='shipped'),
7     Row(order_id=3, customer_id=101, order_date=date(2026, 1, 7), amount=240.00, status='pending'),
8     Row(order_id=4, customer_id=103, order_date=date(2026, 1, 8), amount=18.99, status='cancelled'),
9     Row(order_id=5, customer_id=104, order_date=date(2026, 1, 9), amount=999.95, status='shipped'),
10 ]
11 df = spark.createDataFrame(rows)
12 df.show()
```

```
1 df.write.mode('overwrite').format('delta').saveAsTable('orders_bronze')
2 print('Wrote orders_bronze to the Lakehouse.')
```

Fabric — Data Pipeline

Wait → Run Notebook (design), and a Succeeded run.



Recent runs for pl_demo_ingest

Activity name	Start time	Submitted by	Status
pl_demo_ingest	06/26/2026, 12:42 PM	MOD Administrator	Succeeded

Azure — Fabric capacity

The billable Microsoft.Fabric/capacities resource (F2), deployed by Bicep.

 **fabasc93lr8u**
Fabric Capacity





Diagnose issues with this resource

Show performance trends for this resource

What metric dimensions are supported for this resource

Search

⌵ ⌵

⏸ Pause

🔄 Refresh

➡ Move ⌵

🗑 Delete

Overview

Activity log

Access control (IAM)

Tanzu

Essentials

Resource group (move) : [rg-fabric-as-code](#)

Status : Active

Location : West US 3

Resource name : fabasc93lr8u

SKU : F2

What you can build on this

- Medallion data platform — Lakehouse (bronze/silver/gold) + Spark notebooks
- SQL warehousing — Warehouse with versioned schema, tables & stored procedures
- Orchestration — Data Pipelines chaining notebooks, copies & procs
- Promotion — Git integration + deployment pipelines (dev → test → prod)
- Governance as code — capacity sizing, admins and RBAC in source control

Get started

- `git clone https://github.com/<owner>/fabric-as-code`
- `cp .env.example .env` (fill in tenant / subscription / capacity)
- `pwsh ./scripts/powershell/deploy-all.ps1`
- ...or `./scripts/bash/deploy-all.sh`
- Tear it all down with 99-teardown.